NeuroKinematics

Physical Robot Integration & Execution Architecture Extension

Engineering Architecture & Roadmap Extension Report

---

1. Dokümanın Amacı

Bu dokümanın amacı, mevcut **NeuroKinematics geliştirme roadmap'ini değiştirmeden**, sistemin fiziksel robotlarla gerçek zamanlı olarak çalışabilmesi için gerekli yeni mühendislik katmanlarını tanımlamaktır.
Bu çalışma yeni NeuroKinematics mimarisini

sıfırdan tanımlamamaktadır.
Mevcut:

- Foundations
- Core
- Hybrid
- Studio

fazları ve bu fazlarda geliştirilmiş/geliştirilecek bileşenler korunacaktır.
Ayrıca roadmap'e daha önce eklenen:

- Multi-Robot / Generic Robot desteği
- Trajectory Generation Layer
- Motion Planning Layer

de aynı şekilde korunacaktır.
Bu raporun kapsamı **Motion Planning Layer sonrasında NeuroKinematics'in fiziksel robot yürütme yeteneğine ulaşması için gerekli ilave geliştirme katmanlarını** tanımlamaktır.
Dolayısıyla aşağıdaki başlıklar:

mevcut fazların alternatifi değil, NeuroKinematics'in mevcut mimarisinin devamı niteliğindeki genişleme aşamalarıdır.

---

2. Roadmap Koruma Prensibi

NeuroKinematics geliştirilirken fazların sorumluluklarının birbirine karıştırılmaması temel mimari prensip olacaktır.
Mevcut araştırma ve yazılım geliştirme fazları korunmalıdır.
Özellikle:
Foundations
matematiksel ve deneysel güvenilirlik altyapısını,
Core
Neural IK çekirdeğini,
Hybrid

neural ve numerical çözüm mekanizmalarının birleşimini,

Studio

NeuroKinematics'in kullanıcıya sunulan uygulama, görselleştirme ve etkileşim katmanını temsil etmeye devam edecektir.

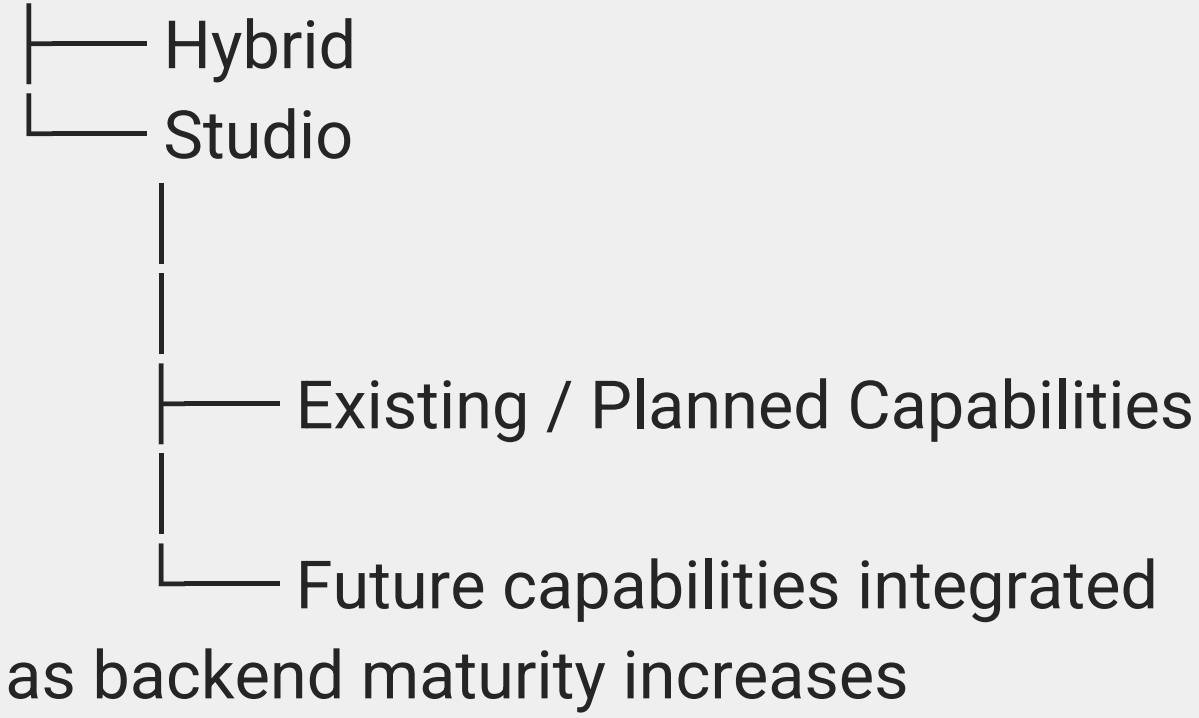
Yeni fiziksel robot katmanları bu fazların görevlerini üstlenmeyecek veya mevcut fazları yeniden gerçekleştirmeyecektir.

---

3. Genişletilmiş Roadmap Mantığı

Roadmap aşağıdaki mantıkla ele alınmalıdır:

```text
MEVCUT NEUROKINEMATICS ROADMAP
│
├── Foundations
├── Core
```



as backend maturity increases

Mevcut roadmap üzerine eklenen mühendislik yetenekleri:

```text
Generic / Multi-Robot Architecture
        ↓
Trajectory Generation
        ↓
Motion Planning
        ↓
_____________________________________
_____________________________
BU RAPORUN BAŞLANGIÇ NOKTASI
_____________________________________
```

↓

Physical Execution Architecture

Bu noktadan sonra aşağıdaki yeni geliştirme başlıkları eklenmelidir.

---

4. PE-01 — Digital Twin Execution Validation Layer

4.1 Amaç

Motion Planning tarafından oluşturulan hareketlerin fiziksel robota aktarılmadan önce robotun dijital modeli üzerinde doğrulanmasını sağlayan yürütme öncesi doğrulama katmanıdır. Bu katman yalnızca görselleştirme amacı taşımamalıdır.

Digital Twin, fiziksel execution öncesindeki doğrulama otoritesi olarak değerlendirilmelidir.

4.2 Temel Gereksinimler

DT-REQ-01 — Common Robot Model

Simülasyon, NeuroKinematics ve fiziksel robot mümkün olduğunca aynı robot model tanımını kullanmalıdır.
Robot geometrisi için tek kaynak yaklaşımı uygulanmalıdır.
Örneğin:

```text
RobotModel
    |
    ├── NeuroKinematics
    ├── Motion Planning
```

```
├── Simulation
└── Physical Robot Configuration
```

DT-REQ-02 — Robot State Synchronization

Fiziksel robotun ölçülen eklem durumları:

```text
q_actual
```

dijital ikize aktarılabilmelidir.
Dijital robotun başlangıç konfigürasyonu teorik başlangıç pozisyonundan değil, mümkün olduğunca fiziksel robotun gerçek durumundan alınmalıdır.

DT-REQ-03 — Planned Motion Simulation

Motion Planning / Trajectory Generation tarafından oluşturulan:

```text
q(t)
dq(t)
ddq(t)
```

hareket profili dijital ikiz üzerinde yürütülebilmelidir.

DT-REQ-04 — Execution Validation

En az aşağıdaki kontroller gerçekleştirilmelidir:

- joint position limits,

- velocity limits,
- acceleration limits,
- self-collision,
- environment collision,
- trajectory continuity,
- final target error,
- solver validity,
- singularity proximity/risk.

DT-REQ-05 — Execution Decision

Digital Twin katmanı yapılandırılmış bir sonuç üretmelidir:

```text
VALID
VALID_WITH_WARNING
INVALID
```

INVALID sonucu alan trajectory fiziksel execution pipeline’ına geçirilmemelidir.

4.3 Çıktılar

- doğrulanmış trajectory,
- validation report,
- joint-limit report,
- collision report,
- final pose error,
- simulation artifacts,
- execution authorization state.

4.4 Faz Çıkış Kriteri

NeuroKinematics tarafından oluşturulan bir hareket, fiziksel robot bulunmadan tamamen sanal ortamda yürütülebilmeli ve makine tarafından değerlendirilebilir bir validation sonucu üretilebilmelidir.

---

5. PE-02 — ROS 2 Hardware Abstraction & Communication Layer

5.1 Amaç

NeuroKinematics ile fiziksel robot elektroniği arasında robottan bağımsız ve standartlaştırılmış haberleşme katmanı oluşturmaktır.
NeuroKinematics çekirdeği doğrudan:

- STM32,
- CAN mesajları,
- motor driver protokolü,
- encoder protokolü

gibi robot-spesifik ayrıntıları bilmemelidir.

5.2 Referans Mimari

```text
NeuroKinematics
        ↓
ROS 2 Integration
        ↓
ros2_control
        ↓
Robot Hardware Interface
        ↓
Transport Driver
```

```
        ↓
Embedded Controller
```

5.3 Gereksinimler

HW-REQ-01 — Generic Hardware Interface

Fiziksel robot için generic command/state sözleşmesi oluşturulmalıdır.
Temel state verileri:

```text
joint_position
joint_velocity
controller_state
fault_state
```

Temel command verileri:

```text
joint_position_target
joint_velocity_target
trajectory_command
control_state
```

HW-REQ-02 — Hardware Adapter Architecture

Yeni robot eklemek için NeuroKinematics çekirdeğinin değiştirilmesi gerekmemelidir. Robot-spesifik entegrasyon:

```text
Hardware Adapter / Driver
```

üzerinden yapılmalıdır.

HW-REQ-03 — Bidirectional Communication

İletişim çift yönlü olmalıdır.

```text
PC → Robot
commands

Robot → PC
state + feedback + diagnostics
```

HW-REQ-04 — Communication State

En az:

```text
DISCONNECTED
CONNECTING
CONNECTED
READY
EXECUTING
FAULT
```

durumları takip edilmelidir.

HW-REQ-05 — Timestamping

Joint state ve trajectory mesajları zaman bilgisi taşımalıdır.
Gerçek ve planlanan hareket karşılaştırmasında zamansal eşleme yapılabilmelidir.

5.4 Faz Çıkış Kriteri

NeuroKinematics:

1. fiziksel robotun gerçek joint state bilgisini okuyabilmeli,
2. robota joint/trajectory command aktarabilmeli,
3. bunu robot-spesifik ayrıntılardan bağımsız bir interface üzerinden yapabilmelidir.

---

6. PE-03 — Embedded Robot Controller Architecture

6.1 Amaç

Gerçek zamanlı robot kontrolünün PC/ROS 2 tarafına bağımlı olmadan robot üzerindeki gömülü kontrolcü tarafından yürütülmesini sağlamaktır.
Referans platform olarak:
STM32 tabanlı gömülü sistem kullanılabilir.
Ancak NeuroKinematics mimarisi belirli bir MCU ailesine bağımlı olmamalıdır.

6.2 Sorumluluk Ayrımı

NeuroKinematics / PC

Sorumlulukları:

- IK,
- Motion Planning,

- Trajectory Generation,
- Simulation,
- high-level execution management,
- visualization,
- diagnostics.

Embedded Controller

Sorumlulukları:

- gerçek zamanlı motor command execution,
- encoder acquisition,
- joint state computation,
- homing,
- limit handling,
- actuator enable/disable,
- watchdog,
- fault state management.

6.3 Gereksinimler

EMB-REQ-01 — Joint State Acquisition

Tüm robot eklemlerinin gerçek pozisyonları ölçülebilmelidir.

EMB-REQ-02 — Motor Command Interface

Embedded controller:

- position,
- gerekirse velocity,
- uygun mimaride torque/current

setpointlerini işleyebilmelidir.

EMB-REQ-03 — Homing

Robot için tanımlı ve tekrar üretilebilir home/ reference mekanizması bulunmalıdır.

EMB-REQ-04 — Configuration

En az:

- encoder resolution,
- encoder direction,
- gear ratio,
- joint direction,
- zero offset,
- motor identification

konfigürasyonu desteklenmelidir.

EMB-REQ-05 — Controller State Machine

Önerilen minimum state yapısı:

```text
BOOT
↓
INITIALIZATION
↓
HOMING
↓
READY
↓
EXECUTING
↓
STOPPED

FAULT
```

EMB-REQ-06 — Communication Watchdog

PC/ROS 2 bağlantısının kaybolması durumunda robotun kontrolsüz davranmaması gömülü katmanda garanti edilmelidir.

6.4 Haberleşme

İlk araştırma/prototip seviyesinde:

- CAN,
- CAN-FD,
- Ethernet

değerlendirilebilir.
Daha ileri ürünleşme aşamalarında transport

katmanı değiştirilebilir olmalıdır.

6.5 Faz Çıkış Kriteri

STM32 tabanlı referans robot controller:

- joint state üretebilmeli,
- high-level command alabilmeli,
- robot actuatorlerini kontrol edebilmeli,
- fault/connection durumlarını dış sisteme aktarabilmelidir.

---

7. PE-04 — Closed-Loop Physical Trajectory Execution

7.1 Amaç

Planlanan hareketin fiziksel robot tarafından gerçekten takip edildiğini gözlemleyen ve execution sürecini kapalı çevrim olarak yöneten katmanın oluşturulmasıdır.
Bu katman NeuroKinematics'i:

trajectory üreten sistemden

şuna dönüştürür:

trajectory execution sonuçlarını gözlemleyen robot-control platformu.

7.2 Temel Model

Planlanan hareket:

```text
```

Robot tarafından ölçülen gerçek hareket:

```text
q_actual(t)
```

Tracking error:

```text
e(t) = q_commanded(t) - q_actual(t)
```

7.3 Gereksinimler

- commanded trajectory tracking,
- actual trajectory acquisition,
- tracking-error calculation,
- execution progress,

- trajectory completion detection,
- target tolerance,
- excessive-error detection,
- execution abort,
- controlled stop,
- fault propagation,
- execution metrics logging.

7.4 Sonuç Metrikleri

Her fiziksel execution için en az:

- mean tracking error,
- maximum tracking error,
- final joint error,
- Cartesian final error,
- duration,

- aborted/completed state,
- controller faults

kaydedilmelidir.

7.5 Faz Çıkış Kriteri

NK yalnızca:

> “Robota şu hareketi yapmasını söyledim.”

diyememeli.
Aynı zamanda:

> “Robotun bu hareketi ne ölçüde gerçekleştirdiğini biliyorum.”

diyebilmelidir.

---

8. PE-05 — Robot Calibration & Sim-to-Real Alignment

8.1 Amaç

Nominal robot modelinin fiziksel robot gerçekliği ile uyumlandırılmasıdır.
URDF geometrisinin doğru olması tek başına yeterli kabul edilmemelidir.

8.2 Calibration Alanları

Joint Calibration

- encoder zero,
- encoder scale,
- encoder direction,

- gear ratio,
- homing offset.

Kinematic Calibration

- link dimensions,
- joint-axis alignment,
- base transform,
- TCP transform,
- tool offset.

Robot Configuration Identification

Fiziksel robot konfigürasyonunun NK robot modeli ile birebir eşleştiği doğrulanmalıdır.

8.3 Sim-to-Real Metrics

En az:

```text
joint error
position error
orientation error
repeatability
```

ölçülmelidir.

8.4 Versioning

Calibration sonucu:

```text
robot_revision
URDF_hash
calibration_version
date
```

ile ilişkilendirilmelidir.
Robot geometrisi değişirse eski calibration otomatik olarak geçerli kabul edilmemelidir.

8.5 Faz Çıkış Kriteri

Aynı trajectory:

- simülasyonda,
- fiziksel robotta

çalıştırıldığında ölçülen farklar raporlanabilir ve tanımlanan mühendislik toleransları içinde olmalıdır.

---

9. PE-06 — Hardware-in-the-Loop Validation & Fault Management

9.1 Amaç

Fiziksel robotu tam execution moduna almadan önce NK → ROS 2 → embedded controller → hardware zincirinin hata durumlarında kontrollü davranış gösterdiğini doğrulamaktır.

9.2 HIL Gereksinimleri

En az aşağıdaki senaryolar test edilmelidir:

- controller disconnect,
- ROS 2 communication loss,
- delayed state feedback,

- corrupted message,
- encoder missing,
- inconsistent encoder value,
- joint limit request,
- trajectory abort,
- embedded controller restart,
- PC restart,
- NK process crash,
- driver fault.

9.3 Fault Management

Hatalar sınıflandırılmalıdır:

```text
INFO
WARNING
RECOVERABLE_FAULT
```

```
CRITICAL_FAULT
```

Her fault için tanımlı:

- detection,
- propagation,
- robot response,
- reset procedure

bulunmalıdır.

9.4 Fail-Safe Prensibi

High-level yazılım kaybı robotun kontrolsüz harekete devam etmesine neden olmamalıdır. Güvenli duruma geçiş yalnızca NeuroKinematics Studio üzerindeki bir komuta bağlı olmamalıdır.

9.5 Faz Çıkış Kriteri

Kontrollü hata enjeksiyon testlerinde sistem öngörülebilir ve dokümante edilmiş davranış göstermelidir.

---

10. PE-07 — Physical Robot Safety Architecture

10.1 Amaç

Araştırma prototipinden gerçek fiziksel robot platformuna geçişte güvenlik sorumluluklarının ayrı bir mühendislik katmanı olarak ele alınmasıdır.
Safety yalnızca yazılım özelliği olarak

değerlendirilmemelidir.

10.2 Minimum Gereksinimler

- physical emergency stop,
- actuator disable,
- hardware limits,
- software limits,
- controller watchdog,
- communication timeout,
- fault-triggered stop,
- safe startup,
- safe shutdown,
- controlled reset,
- motor driver fault handling.

10.3 Mimari Prensip

```text
Studio Emergency Stop
```

kullanışlı bir kullanıcı fonksiyonudur ancak:

```text
Physical Safety E-Stop
```

mekanizmasının yerini almamalıdır.
Safety-critical davranış mümkün olduğunca high-level PC yazılımından bağımsız tasarlanmalıdır.

---

11. STUDIO-EXT — Physical Robot

Operations Extension

11.1 Önemli Roadmap Notu

Bu başlık **yeni bir Studio fazı değildir.**
NeuroKinematics roadmap’inde halihazırda bulunan **Studio fazının ilerleyen sürümde genişletilmesidir.**
Studio'nun mevcut:

- visualization,
- robot interaction,
- solver interaction,
- simulation,
- model management

sorumlulukları korunacaktır.
Physical Robot Integration katmanları olgunlaştıkça Studio bunlara kullanıcı arayüzü sağlamalıdır.

11.2 Gereksinimler

Studio üzerinden:

Robot Management

- robot selection,
- connect/disconnect,
- robot readiness,
- calibration status,
- controller state,
- hardware compatibility.

Live Robot State

- actual joint positions,
- target joint positions,
- joint velocities,

- TCP pose,
- tracking errors,
- fault state.

Execution Pipeline

Studio aşağıdaki iş akışını desteklemelidir:

```text
TARGET
↓
SOLVE
↓
PLAN
↓
VALIDATE
↓
SIMULATE
↓
AUTHORIZE
↓
EXECUTE
↓
```

MONITOR

Execution Monitoring

- planned robot state,
- actual robot state,
- progress,
- errors,
- warnings,
- stop,
- abort.

11.3 Mimari Kısıt

Studio doğrudan:

- CAN driver,
- STM32 protocol,

- servo communication

implementasyonlarını içermemelidir.
Studio yalnızca standardize edilmiş NK/ROS 2 backend servislerini kullanmalıdır.

---

12. PE-08 — Deployment, Reliability & Productization Architecture

12.1 Amaç

TK56 üzerinde doğrulanan sistemin tek robota özel bir akademik prototip olarak kalmasını önlemek ve gelecekte farklı robot sistemlerine uygulanabilecek bir ürün mimarisine hazırlanmasını sağlamaktır.

12.2 Robot Independence

Ürün seviyesinde aşağıdaki ayrım korunmalıdır:

```text
NeuroKinematics Core
        │
        ├── Robot Model Plugin
        ├── Hardware Adapter
        ├── Calibration Profile
        └── Controller Configuration
```

Yeni robot için NK Core değiştirilmemelidir.

12.3 Developer Interface

Gelecekte aşağıdaki extension interface'ler değerlendirilebilir:

- Robot Model SDK
- Hardware Driver SDK
- Solver Plugin API
- Motion Planner Plugin API
- Simulator Adapter API
- Calibration API

12.4 Reliability

Ürünleşme öncesinde:

- automated regression,

- endurance testing,
- structured diagnostics,
- crash recovery,
- version compatibility,
- dependency management,
- telemetry,
- deployment validation

gibi mühendislik gereksinimleri eklenmelidir.

12.5 Product Boundary

NeuroKinematics'in ticari çekirdeği:

> motor sürücü firmware'i üretmek

olmamalıdır.

Daha doğru platform sınırı:

> farklı manipülatör modellerini tanımlayan, IK ve motion planning yapan, simülasyonla

doğrulayan ve standart donanım arayüzleri üzerinden fiziksel robot execution'ını yöneten robot-agnostic engineering and control platform.

olmalıdır.

---

13. Güncellenmiş Sistem Katmanları

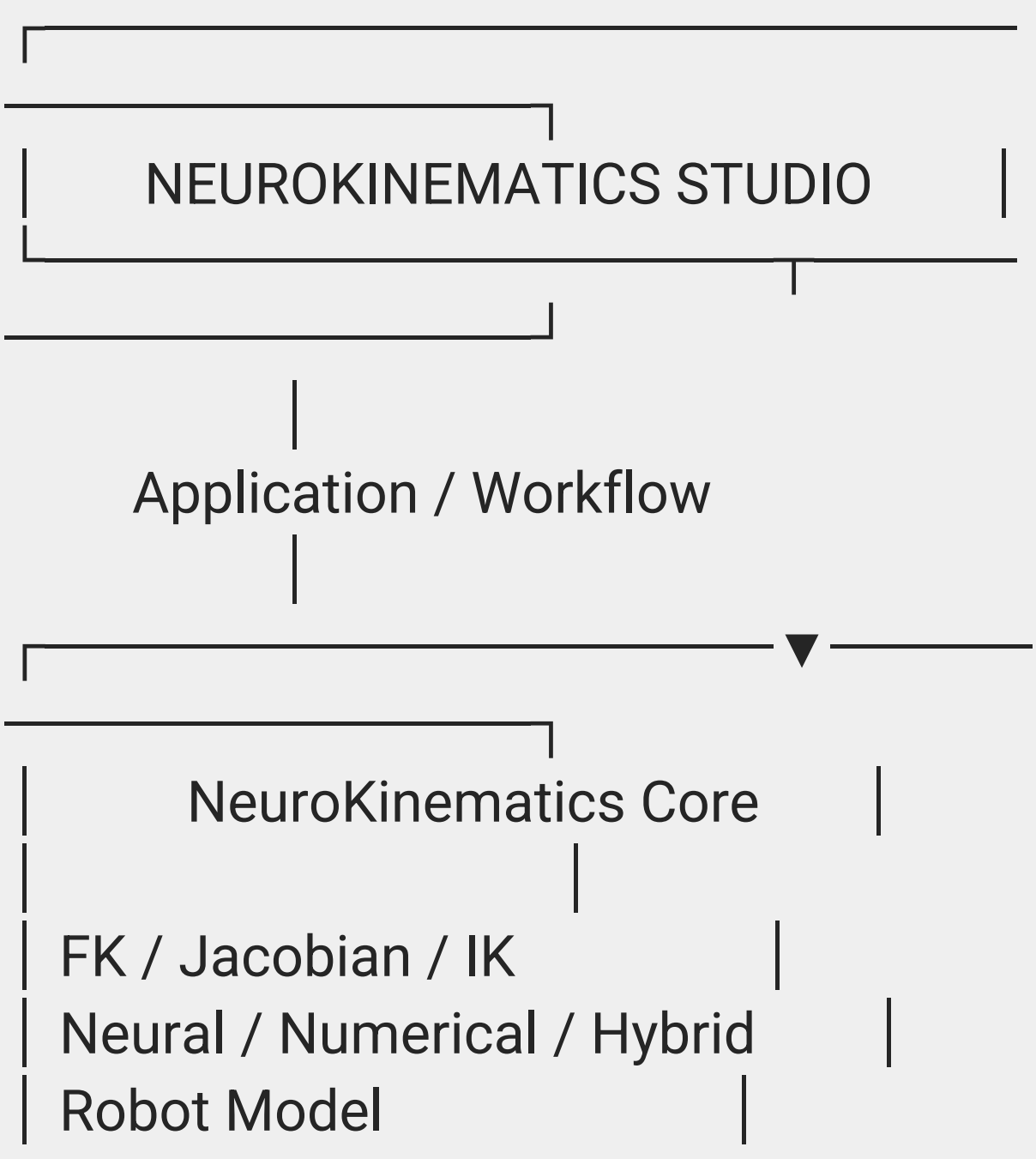
Roadmap fazları ile runtime mimarisi birbirinden ayrılmalıdır.
Roadmap:

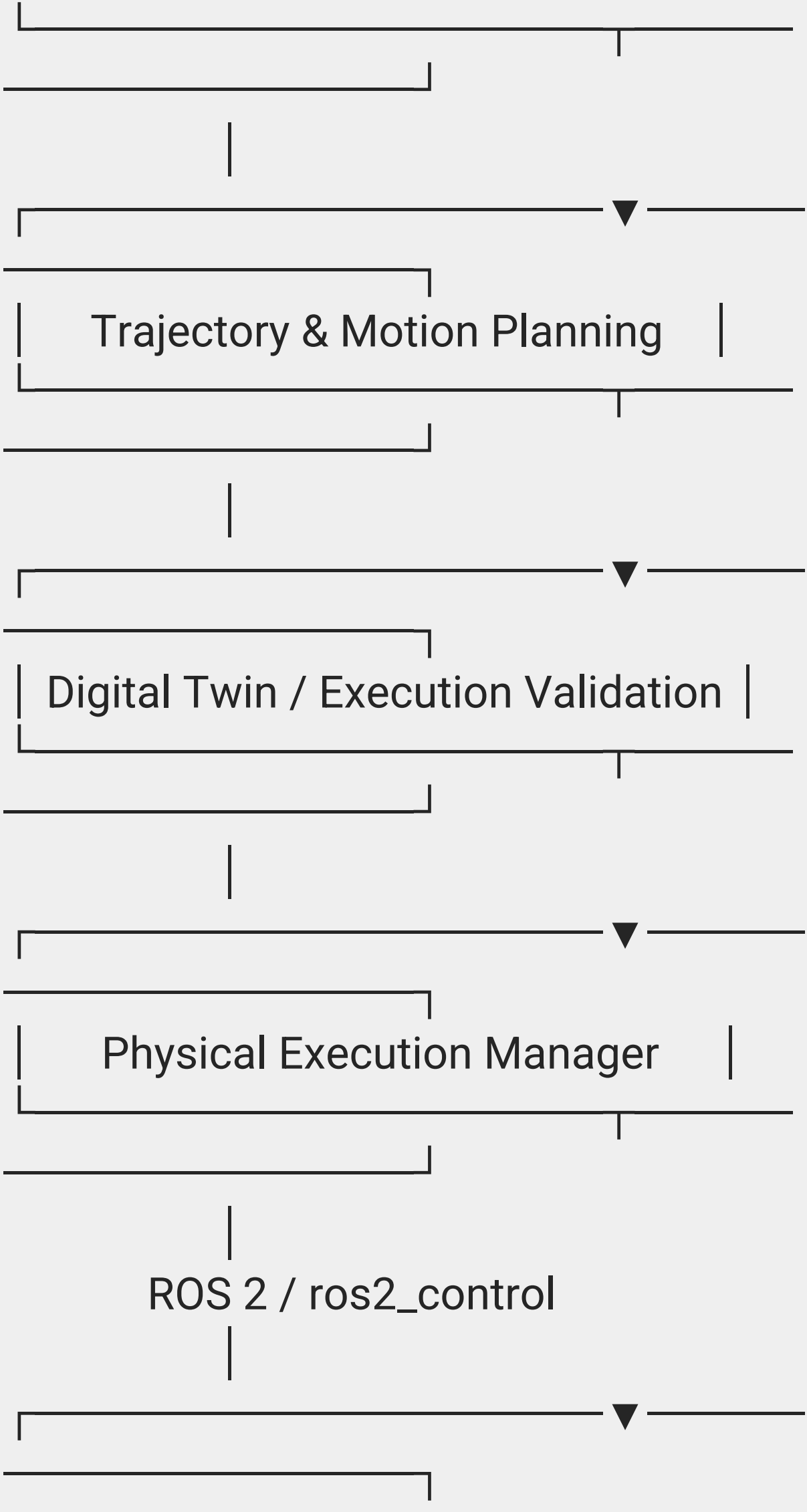
```text
Foundations
Core
Hybrid
Studio
```

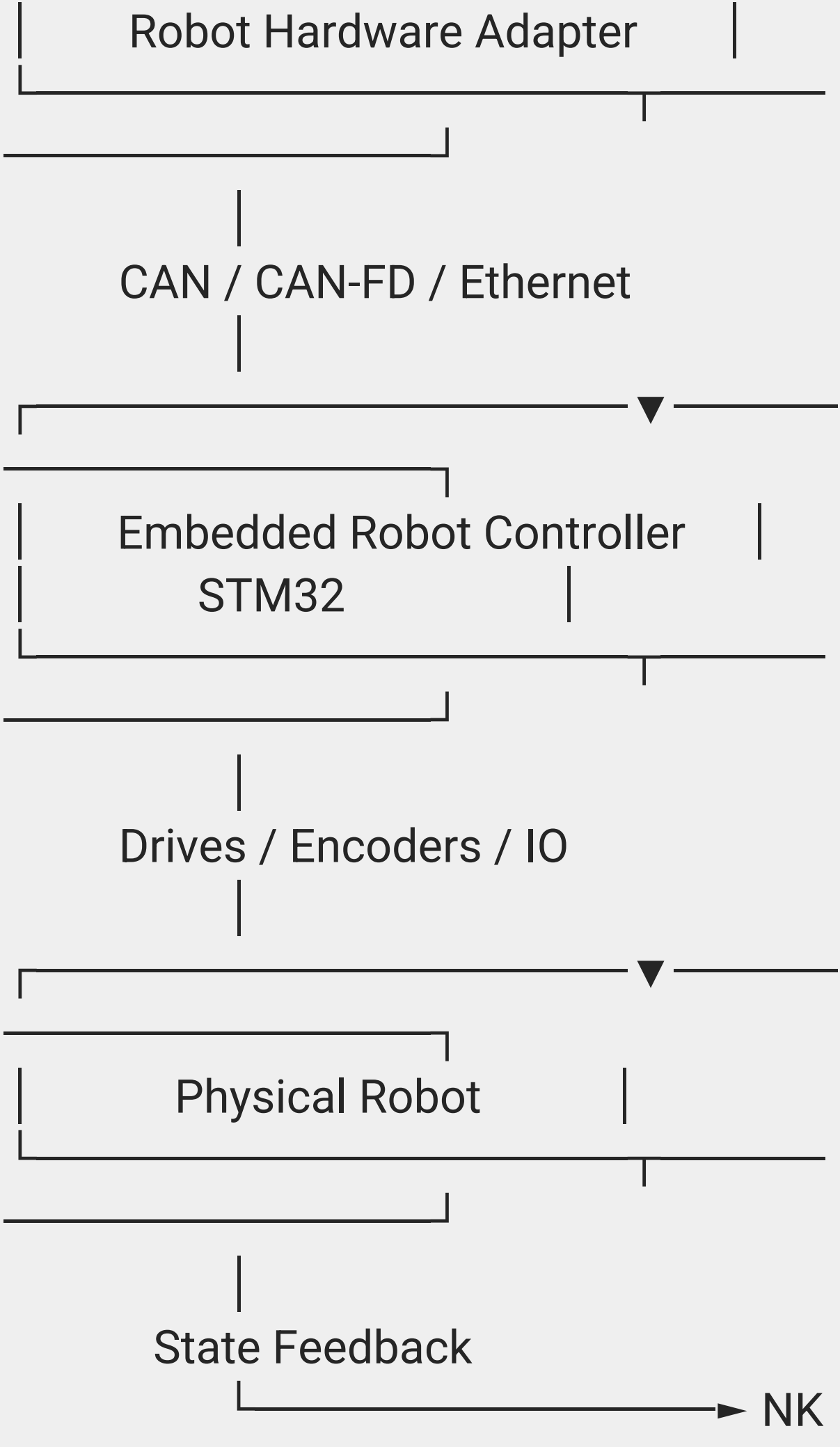
+
Generic Robot Extensions
Trajectory Generation
Motion Planning
Physical Execution Extensions

Runtime mimarisi ise nihai olarak:

text




Trajectory & Motion Planning
Digital Twin / Execution Validation
Physical Execution Manager
ROS 2 / ros2_control


Robot Hardware Adapter
CAN / CAN-FD / Ethernet
Embedded Robot Controller
STM32
Drives / Encoders / IO
Physical Robot
State Feedback
NK

---

14. Temel Tasarım İlkeleri

Geliştirme boyunca aşağıdaki mimari prensipler korunmalıdır.

P-01 — Separation of Concerns

IK, planning, simulation, hardware communication ve motor control birbirinden ayrılmalıdır.

P-02 — Hardware Independence

NK Core hiçbir STM32, CAN veya motor driver implementasyonuna doğrudan bağımlı olmamalıdır.

P-03 — Robot Independence

Yeni bir robot eklenmesi NK Core'un yeniden yazılmasını gerektirmemelidir.

P-04 — Simulation Before Execution

Fiziksel execution öncesi validation pipeline bulunmalıdır.

P-05 — Feedback Required

Fiziksel robot yalnızca command recipient olarak değil state-producing system olarak modellenmelidir.

P-06 — Safety Outside Application Layer

Safety yalnızca Studio veya ROS 2 sürecine bağlı olmamalıdır.

P-07 — Reproducibility

Robot model, firmware, calibration, solver ve trajectory sürümleri izlenebilir olmalıdır.

P-08 — Incremental Validation

Her fiziksel entegrasyon katmanı ayrı kabul testinden geçmeden sonraki katmana geçilmemelidir.

---

15. Fazlar Arası Bağımlılık

Yeni fiziksel execution başlıkları sıralı şekilde geliştirilmelidir.

```text
Motion Planning
      ↓
```

PE-01 Digital Twin Validation

↓

PE-02 ROS 2 Hardware Abstraction

↓

PE-03 Embedded Controller

↓

PE-04 Closed-Loop Execution

↓

PE-05 Calibration / Sim-to-Real

↓

PE-06 Hardware-in-the-Loop

↓

PE-07 Safety Architecture

↓

Studio Physical Robot Extension

↓

PE-08 Productization

Bazı çalışmalar paralel yapılabilse de acceptance gate'ler bu sırayı takip etmelidir.

---

16. NeuroKinematics'in Nihai Teknik Sınırı

Bu genişlemeler tamamlandığında NeuroKinematics aşağıdaki uçtan uca zinciri yönetebilecek mimariye sahip olmalıdır:

```text
Robot Model
↓
Robot State
↓
Target Definition
↓
Inverse Kinematics
↓
Motion Planning
↓
Trajectory Generation
↓
Simulation
↓
```

Validation
↓
Physical Execution
↓
Feedback
↓
Tracking Evaluation

Ancak NK'nın kendisinin doğrudan:

- PWM üretmesi,
- MOSFET sürmesi,
- encoder interrupt yönetmesi,
- motor current control yapması

beklenmemelidir.
Bu görevler embedded/drive katmanının sorumluluğunda kalmalıdır.

---

17. Referans Fiziksel Platform

İlk fiziksel validation platformu olarak:
Arda TK56
kullanılabilir.
TK56'nın rolü yalnızca demo robot olmak değil, NeuroKinematics'in:

- custom robot onboarding,
- simulation,
- planning,
- ROS 2 integration,
- physical execution,
- closed-loop feedback,
- sim-to-real

iddialarını uçtan uca doğrulayan **reference physical platform** olmaktır.
KUKA KR6 R900 sixx ise sistemin kinematik/algoritmik referans robotlarından biri olarak

korunabilir.

Bu ayrım önemlidir:

```text
KUKA KR6
→ algorithmic/reference validation

Arda TK56
→ full physical integration validation
```

---

18. Stratejik Nihai Hedef

NeuroKinematics uzun vadede belirli bir robotun kontrol yazılımı olarak tasarlanmamalıdır.

Hedef mimari:

Robot üreticisi veya geliştiricisi kendi manipülatör modelini NeuroKinematics'e

tanımlar; NK robotun kinematik yapısını kullanır, uygun solver ve planning pipeline'ını çalıştırır, hareketi simülasyonda doğrular ve standart bir hardware integration layer üzerinden fiziksel sisteme uygular.

Bu yaklaşım NeuroKinematics'in akademik araştırma projesinden tekrar kullanılabilir robotik mühendislik platformuna evrilmesini sağlar.

---

19. Roadmap Değişiklik Politikası

Bu doküman uygulanırken mevcut roadmap maddeleri silinmemeli veya fiziksel robot entegrasyonu uğruna yeniden tanımlanmamalıdır.
Yeni başlıklar açıkça:

```text
ROADMAP EXTENSION
```

olarak işaretlenmelidir.
Her yeni başlık için ayrıca:

- objective,
- scope,
- dependencies,
- functional requirements,
- non-functional requirements,
- tests,
- acceptance criteria,
- generated artifacts,
- regression requirements

tanımlanmalıdır.
Bu rapor üst seviye mimari gereksinim dokümanıdır.
Her PE aşaması uygulamaya alınırken ayrıca

ayrıntılı implementation specification ve Codex/ engineering prompt hazırlanmalıdır.